

深度学习训练性能优化:原理、技术与工具

介 飞^{1,2}, 张海俊^{1,2}, 汪锦想¹

- (1. 科大讯飞股份有限公司 核心研发平台, 安徽 合肥 230088;
2. 中国科学技术大学 计算机科学与技术学院, 安徽 合肥 230027)

摘要: 深度学习训练中的模型与数据规模快速增长, 导致模型训练成本不断上涨, 如何高效完成模型训练, 是深度学习落地的主要挑战之一。分析了通用深度学习训练性能优化的一般原理, 从训练的常规流程出发, 系统性分析与总结了异构计算模型中常见的训练性能优化技术。按技术作用阶段, 可分为数据准备、前/反向传播、梯度同步、参数更新阶段的性能优化技术; 从计算机组成角度, 这些技术又可划分为计算、通信、内存、IO 等方面的优化。同时, 介绍了常用的深度学习性能分析与可视化工具, 旨在为相关从业人员开展深度学习训练时的性能优化提供参考。

关键词: 训练性能; 深度学习; 前向计算; 反向传播; 参数更新; 负载均衡

DOI: 10.11907/rjdk.241918

中图分类号: TP18

文献标识码: A

开放科学(资源服务)标识码(OSID):

文章编号: 1672-7800(XXXX)0XX-0001-07



Training Performance Optimization in Deep Learning: Principles, Techniques and Tools

JIE Fei^{1,2}, ZHANG Haijun^{1,2}, WANG Jinxiang¹

- (1. Research & Development Group, iFLYTEK Co., Ltd., Hefei 230088, China;
2. School of Computer Science and Technology, University of Science and Technology of China, Hefei 230027, China)

Abstract: As the scale of models and data in training tasks of deep learning grows rapidly, the costs associated with model training continue to rise, making efficient model training one of the primary challenges in deploying deep learning solutions. This paper analyzes the general principles of performance optimization in deep learning training. Starting from the typical training workflow, it systematically analyzes and summarizes common performance optimization techniques for heterogeneous computing models. These techniques can be categorized based on their stage of application: data preparation, forward/backward propagation, gradient synchronization, and parameter update. From a computer architecture perspective, these techniques can further be classified into optimizations in computation, communication, memory, and I/O. This paper also introduces commonly used tools for performance analysis and visualization in deep learning, so as to provide a valuable reference for practitioners engaged in optimizing training performance of deep learning.

Key Words: training performance; deep learning; forward computation; backward propagation; parameter update; workload balance

0 引言

深度学习在语音识别、计算机视觉、自然语言处理等领域取得巨大成功^[1-3], 业界推出了各类型专用设备用来加速深度神经网络训练(如 NVIDIA 公司的 GPU、Intel 与

XILINX 公司的 FPGA 以及 Google 公司的 TPU 等设备)。但由于模型与数据规模的爆炸式增长, 尤其是以 OpenAI 引领的大语言模型浪潮以来, 深度神经网络模型的训练成本愈发高昂, 如何提升训练性能, 降低训练成本成为研究的热点之一。本文站在系统性视角, 从深度学习训练的全流程分析与归纳通用的深度学习训练性能优化原理、技术与

收稿日期: 2024-11-21

扫描二维码阅读全文:



基金项目: 安徽省博士后研究人员科研活动经费资助项目(2022B588)

作者简介: 介飞(1991-), 男, 博士, CCF 会员, 科大讯飞股份有限公司研究算法工程师, 科大讯飞股份有限公司与中国科学技术大学联合招收在站博士后, 研究方向为深度学习与数据挖掘; 张海俊(1990-), 男, 科大讯飞股份有限公司高级引擎架构师, 中国科学技术大学博士研究生, 研究方向为深度学习与高性能计算; 汪锦想(1987-), 男, 硕士, 科大讯飞股份有限公司科学家, 研究方向为深度学习与高性能计算。本文通讯作者: 介飞。

工具,供相关从业者参考使用。

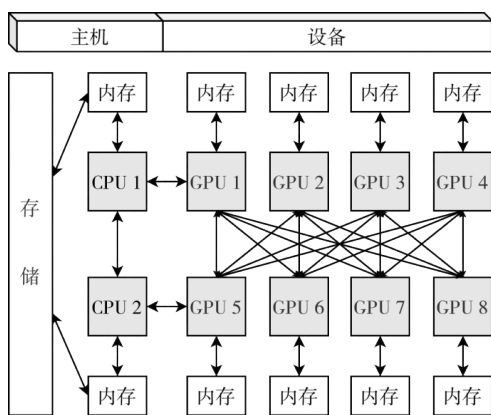


Fig. 1 A diagram of heterogeneous computing architecture

图1 一种异构计算架构示意图

目前,由于面向通用任务的CPU架构对深度学习计算不友好,一般使用专用计算设备进行训练任务加速,因而深度学习训练任务通常是异构计算模型(一种基于CPU与GPU的异构计算架构,如图1所示)。CPU负责训练逻辑的分发,专用计算设备用于计算加速。但由于异构计算模型架构,深度学习训练效率需要通用计算设备(主机侧,如CPU服务器)与专用计算设备(设备侧,如GPU/TPU等)高效配合,才能最大程度地发挥设备性能。因此,要提升深度学习的训练性能,需从系统视角考虑异构计算架构的特点,全面评估计算、通信、内存、IO等对性能的影响。需遵循的原则大致包括:①尽量提升训练任务中高性能计算设备的使用占比,降低设备空闲时间;②尽量提升有效计算在总训练时间中的占比,有效计算包括模型的前/反向传播、参数更新等;③尽量降低空闲/通信/无效计算在总训练时间中的占比,包括梯度同步、算子分发耗时等。

1 问题与挑战

大规模深度学习系统完成模型训练通常包括数据准备、前/反向传播、梯度同步、参数更新,上述步骤涉及数据在不同类型设备之间的计算、通信与IO调度。大规模深度学习任务通常涉及异构计算设备,不同设备的性能存在差异,实现异构计算设备的高效协同是改进深度学习训练任务性能的核心原则。

大规模深度学习训练任务通常面临以下问题与挑战:

①训练数据规模大,深度学习训练任务的数据规模数以亿计,读取与处理这些数据成为完成训练任务的第一道障碍,如用于训练大语言模型的Token数据达到千亿级别^[4];②计算所需算力高,目前最大的深度学习模型参数规模已超万亿,完成模型计算需庞大的算力,单一计算设备已无法满足该需求,这成为深度学习模型训练面临的另一大挑战^[5];③通信交换数据多,当前深度学习训练任务通常基于分布式系统,分布式节点之间需进行大量的通信以同步模型的训练状态,这对系统的通信能力提出了较高要求。

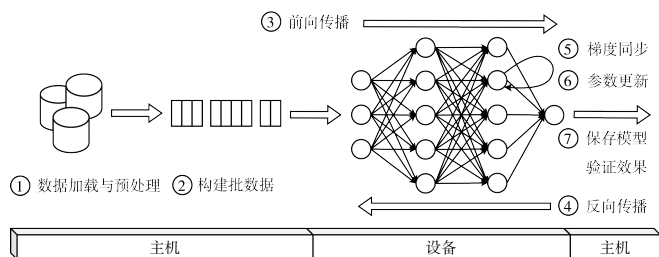


Fig. 2 A schematic diagram of the deep learning training process

图2 深度学习训练流程示意图

本文从系统角度,分析深度学习训练任务中所涉及的数据、算力、通信流程,总结与归纳通用深度学习训练性能优化技术与工具。一个典型的深度学习流程包括数据加载与处理、模型前/反向计算、梯度同步与参数更新等阶段,如图2所示。本文将以该流程为顺序,介绍各阶段可用的性能优化技术、跨多个流程的优化手段以及相关性能分析与可视化工具。

2 数据准备

现有深度学习领域的繁荣建立在大数据基础之上,如何处理与准备庞大的训练数据是完成训练任务的第一步。

2.1 数据处理

数据处理包括数据变换(如特征提取)、归一化、数据增强(如图片裁剪/旋转/翻转、语音加噪/调幅/变速)等操作,该过程可通过多线程/进程并行处理,并利用Cython、Taichi、C/C++等高效工具或语言优化基于Python的处理逻辑,加速执行。

2.2 数据加载

深度学习的核心是随机梯度下降算法,该算法存在对文件系统的大量随机访问,严重降低了系统性能^[6]。数据的随机访问对持久存储设备不友好,会造成IO瓶颈。分批训练是深度学习系统的特征之一,对数据集的一次训练包含多个迭代,每个迭代负责对某批数据进行计算,即数据集提前被划分为多个批次,是固定的。根据这些特性,可开展如下优化:①使用高性能存储,共享存储的数据IO通常存在瓶颈,条件允许时可以考虑更换符合非易失性内存主机控制器接口规范(Non-Volatile Memory Express, NVME)的硬盘存储数据;②设置合理的非统一内存访问(Non-Uniform Memory Access, NUMA)数量与进程绑定到指定CPU核心,充分利用CPU核心与内存访问的亲性和;③可以启动多个进程,读取处理训练数据;④将随机采样限制在固定规模的局部数据中以避免大范围的随机访存,并将数据预取到缓存中供后续迭代访问,TensorFlow与PyTorch框架均采取了该机制;⑤为了保证数据混淆的全局性,可以使用多粒度混淆方案,以大粒度完成全局混淆,以细粒度完成局部混淆,保证局部范围内的数据连续性,以加快读取速度^[7-8];⑥配置优化,在内存允许时,开启框架的数据读取性能优化选项,如PyTorch的pin_memory与

persistent_workers 选项,减少内存操作开销;可以通过设置 OMP_NUM_THREADS 环境变量,优化使用 OpenMP 时的并行计算任务;⑦切换内存分配器,可选的内存分配器有 malloc、Jemalloc、TCMalloc 等。

现代深度学习框架的数据准备逻辑通常由单独的进程完成,当模型计算时间超过数据准备时间时,则当前步的数据准备可以与上一步的模型计算同时进行,即数据准备可以被模型计算时间掩盖,此时优化数据准备逻辑无法获得性能优化。当数据准备无法被模型计算掩盖时,可尝试增大每批数据的大小,增加模型计算时间,以提升数据准备时间被计算掩盖的可能性。

3 模型计算

模型计算包括前向计算、损失计算与梯度的反向传播,其中反向传播一般由深度学习框架自动完成,因而用户一般进行模型前向与损失计算逻辑的优化。

3.1 代码优化

专用计算设备利用并行计算实现矩阵相乘等操作的加速,为了充分利用该优势,需要尽量避免重复计算逻辑的引入。而以矩阵计算的形式代替,如通过改写程序,将某些循环计算逻辑等价替换为矩阵计算,可减少 CPU 的指令分发,提升计算并行性,从而实现模型计算加速。有时,部分模型计算逻辑是冗余的,如卷积操作后如果是批归一化(Batch Normalization),则卷积的偏置项冗余,可以删除。可以降低内存分配与写入、跨设备同步等成本较高的操作,提升效率,如在执行反向传播前,模型需清空上一步的梯度信息,此时可通过设置梯度为 None,而并非对梯度对应内存置零加以实现,这可以减少内存操作,PyTorch 框架可以通过指定 optimizer.zero_grad(set_to_none=True)应用该优化。数据打印、跨设备搬运等操作涉及设备间同步操作,会打断计算流从而影响效率,应尽量避免,如 PyTorch 中应减少 tensor.cpu()、tensor.cuda()等操作。

对于语音识别或机器翻译等输入变长的任务,可通过预先分配较大设备内存的方式,减少内存分配开销与内存碎片产生。

此外,也可以开启高效计算库,如 NVIDIA cuDNN 库提供了多种卷积算法实现,可通过设置 torch.backends.cudnn.benchmark 开启自动算法寻优,加速卷积运算。

Table 1 Computility of NVIDIA GPUs at different precisions
表1 NVIDIA GPU 不同精度算力

型号	性能指标(TFLOPS)			
	FP32	TF32	BF16	FP16
A100 80GB SXM4	19.5	156	312	312
H100 80GB SXM5	67	989	1 979	1 979

3.2 混合精度训练

由于计算加速设备的低精度(FP16/BF16/FP8)算力通常远高于单精度(FP32)算力,如 NVIDIA GPU 设备可以使

Tensor Cores 大幅提升低精度算力(不同精度 GPU 设备算力情况如表 1 所示,数据来自于 NVIDIA 官网),因此可以使用混合精度训练(Mixed Precision Training)^[9-10],通过维护高精度权重副本、损失放缩(Loss Scaling)或者量化技术,降低训练中模型部分结构的权重、激活与梯度^[11]的表示精度,几乎无损地加速模型计算过程。模型计算可分为向量点积、规约、逐点操作等,其中规约与逐点操作为内存受限,计算速度对数值精度不敏感。因此,混合精度训练的加速程度取决于模型计算中向量点积操作的占比,在 NVIDIA Volta GPU 上通常可以取得相对于 FP32 基线 2~6 倍的加速^[9]。

3.3 计算图优化

3.3.1 设备依赖优化

对某些特定算子,输入张量的内存格式会影响计算效率,因此需转换为符合特定规则的内存格式。例如,计算机视觉模型中通道维度处于张量的最后一维时,可在自动混合精度模式下使 Tensor Cores 加速相关计算。GPU 等设备对不同形状张量的计算效率存在差异,可以精心设计模型各计算阶段的张量形状,以充分发挥设备的计算性能,如 Tensor Cores 的最佳性能要求张量满足特定维度是某一数值的整数倍。

3.3.2 算子融合

通过数学计算上的等价替换,将多个算子融合为一个,减少冗余计算,同时减少 CPU 指令下发到设备(GPU/NPU 等)的次数,从而提高性能。例如,通过 PyTorch 的 compile 机制,可将逐元素操作(如 sin()、sigmoid()等)组合为 1 个单一算子^[12-14],可以减少内存访问和 Kenerl 发射次数(程序从 CPU 到专用计算设备的过程称为 Kernel 发射)。

算子融合还可以结合设备特性进行计算流程优化,这一思路最成功的案例是 FlashAttention 系列算法^[15-17],其利用 NVIDIA GPU 不同层次存储的速度差异,通过分块计算(Tilling)机制优化注意力计算流程,减少这一过程中不同层次内存之间的数据搬运,从而大幅提升计算效率。

3.3.3 深度学习编译器

深度学习编译器以各类深度学习框架的模型描述作为输入,生成面向各类深度学习硬件的高效代码实现^[18],自动完成从模型到硬件运算的映射。深度学习编译通常分为前端与后端,其中前端完成模型定义到计算图中间表示的转换,后端完成从高级中间表示到低级中间表示的转换以及设备相关优化。深度学习编译器的前端优化主要是对计算图的优化,包括:①节点级优化,如移除冗余计算节点,当输入为零维张量 A 与固定张量 B 时,则 A 与 B 的加和操作可以用固定张量 B 代替^[18];②块级优化,此类优化涉及多个计算节点,如常量折叠,将常量表达式以其对应的值代替,减少程序运行时的开销^[19];③算子融合,也可通过编译器自动完成算子融合,如 TVM 设计一系列融合规则对各类算子进行融合^[20],对涉及动态形状张量的计算需

设计额外机制进行抽象与处理^[21-22]。

4 梯度同步

在分布式深度学习中,包含多个数据的发送与接收方,发送方与接收方之间的通信方式称为通信原语^[23],梯度同步阶段常用的通信原语包括 Broadcast、AllReduce 等。

4.1 通信算法优化

可通过优化集合通信原语的实现逻辑,以减少梯度同步阶段的通信量,充分利用通信带宽,从而改善通信效率,如 AllReduce 高效实现 RingAllReduce^[24],可实现通信量与分布式设备数无关。通信框架 Horovod 基于 NCCL 与 MPI,采用梯度融合(Tensor Fusion),进一步提升了通信效率^[25]。Blink 框架则优化了通信链路的选择策略,用生成树拓扑代替 RingAllReduce 的环状拓扑,以充分利用设备之间的通信链路,从而改进训练中的通信效率^[26]。

4.2 梯度压缩

可以通过稀疏^[27]、量化^[28-29]等技术对梯度信息进行压缩,以减少分布式训练中梯度同步时的通信数据量。其中,梯度稀疏是指仅同步重要的梯度信息^[27],如超过一定阈值的梯度不进行同步。梯度压缩是指利用低精度数值类型表示梯度信息,降低梯度同步时的数据量^[28]。

4.3 计算通信掩藏

深度神经网络的分层结构使得反向传播时的通信与计算重叠成为可能,网络的某一层梯度完成计算后可立即开始梯度的跨设备同步,即无等待反向传播(Wait-Free Backward Propagation, WFBP)^[30-32],当该层梯度的同步时间大于其下一层梯度的计算时间时,则梯度的通信成本可被完全掩藏。可对梯度进行分桶,当桶内梯度计算好后,即可启动梯度同步,PyTorch 框架可通过分布式训练后端 DDP 的参数 bucket_cap_mb 进行桶大小设置。

4.4 降低梯度更新频率

可在一定训练迭代步内仅更新本地模型参数,以减少设备间梯度同步次数,从而减少通信,提升训练吞吐。如异步 SGD(Asynchronous SGD)^[3-34]算法中设备计算出随机梯度后直接更新本地模型,不再等待所有设备完成梯度计算;而 BMUF 系列算法则周期性地梯度同步,一定程度上降低了梯度同步成本^[35-36]。

5 参数更新

5.1 高效优化器

常见的深度学习模型通常使用 SGD、Adam^[37]或 AdamW^[38]优化器,其中 Adam 与 AdamW 优化器的收敛速度较快,但它们内存占用较大且计算成本较高。因此,一些改进的优化器被提出用于大语言模型等方面的训练,如仅跟踪动量的优化器 Lion^[39]通过寻优得到,较 Adam 优化器

占用内存更少,而 Sophia^[40]是一款轻量的二阶优化器,通过梯度与估计 Hessian 的动态平均、裁剪等手段,可将训练速度翻倍。

5.2 融合优化器

通常,优化器在模型更新阶段会频繁进行算子分发,而这些操作通常逻辑相同,只是处理数据不同,因此可以对优化器涉及算子进行融合,从而提高性能^[41]。

5.3 降低 Kernel 发射占比

Kernel 发射过程除设备计算时间外,还有额外延迟,如 CPU 端函数调用等的耗时,这部分延迟是无效计算且与数据大小无关。因此,可考虑尽可能扩大批数据,降低训练过程中无效计算占比,提升计算设备利用率,从而提升训练效率。

6 系统优化

还有一些系统性的深度学习效率优化技术与手段,它们通常不止作用于深度学习的某个流程,而是作用于多个阶段,甚至是全局性的,涉及对程序调度、通信、内存、IO 等方面的优化,下文将对这些技术进行详细介绍。

6.1 进程优化

在深度学习训练中,可能因为大量进程的启动与销毁而频繁触发程序的垃圾回收机制,但该操作代价较大,会在一定程度上影响 CPU 性能,导致训练效率波动。可根据硬件资源合理配置垃圾回收机制,减少垃圾回收对训练性能的影响,Python 可通过 gc.collect() 与 gc.set_threshold() 进行配置。

6.2 增大批数据规模

通过数据并行增加计算节点,或通过梯度累积(Gradient Accumulation),可等价扩大一次训练迭代中的批数据规模,从而减少迭代次数,缩短训练时间。但过大的批数据规模可能导致模型收敛困难,可通过精细调节参数更新策略以缓解该问题,如逐层自适应学习率缩放(Layer-wise Adaptive Rate Scaling, LARS)策略根据每层权重范数分别调整更新程度,使模型收敛更稳定^[42-43]。

6.3 内存优化

在模型训练中,计算设备中除各层激活占用内存外,模型训练内存主要包括:优化器状态、梯度和参数。可通过分割这 3 部分数据至各数据并行进程,消除数据并行算法的冗余内存占用,此时通信成本增加不超过 50%,该算法即为零冗余优化器(Zero Redundancy Optimizer, ZeRO)算法^[44],ZeRO 算法配合激活重计算^[45],可最大限度地降低内存占用。当相关模型的瓶颈在于设备内存限制而导致计算吞吐不高时,可考虑使用该类方法增大批数据大小,以充分发挥计算加速设备的算力,从而减少训练时间。完全分片数据并行(Fully Sharded Data Parallel, FSDP)算法使用类似 ZeRO 的思路,设计了混合切片策略,允许用户指

定用于梯度、模型参数与优化器状态的节点或进程^[46]。

深度学习框架在特定模式下(如 PyTorch 的 eager 模式)设备内存的申请与释放会产生较多碎片,此时若模型训练所需内存不够则会触发内存整理操作,以得到大块连续内存。但该操作成本较高,频繁触发会影响训练效率,可通过相关配置优化内存申请与释放,如 PyTorch 框架提供环境变量 `PYTORCH_CUDA_ALLOC_CONF`,通过合理设置可以减少内存碎片。

6.4 负载均衡

深度学习中的负载均衡是指分布式集群中每一个工作节点之间工作负载与其工作能力的均衡^[47]。负载均衡通常发生在序列模型训练中,如语音识别、机器翻译、语言模型等,此类模型的单步计算时间通常与输入的序列数据长度有关,若多个设备上的序列数据长短不一,则其他设备需等待处理最长序列的设备完成计算,此时发生计算负载不均衡。避免负载不均衡的方式很多,核心思路是尽量平均分配工作负载,可通过序列切分、分桶或排序的方式,使不同设备的序列数据长度尽量接近,以减轻负载不均衡情况。除划分数据使负载均衡外,还需考虑内存和通信负载均衡,即各设备内存占用与通信数据量需大致平衡。深度学习训练任务中负载均衡与不均衡情况如图 3 所示,负载不均衡时设备在梯度同步前的计算时长不同,负载小的设备需等待负载大的设备;负载均衡后,无效等待时间缩短,训练效率提升。

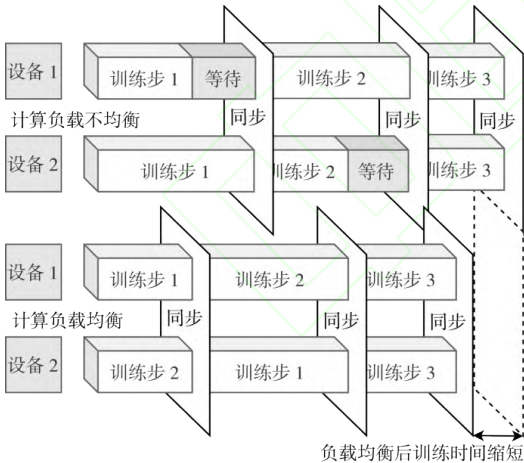


Fig. 3 The impact of load balance on training efficiency
图3 负载均衡对训练效率的影响

从计算机组成角度,优化技术可进一步分为对计算、通信、内存与 IO 的优化。综上,主要优化技术总结如表 2 所示。

相关开源工具实现了部分训练优化技术,使用较广泛的工具、其使用的优化技术及特性如表 3 所示。

7 性能分析与可视化工具

性能分析工具用于分析和优化模型的执行过程,本文

Table 2 Summary of various training optimization techniques by training process and domain

优化技术	涉及训练流程					所属领域			
	数据准备	前向传播	反向传播	梯度同步	参数更新	计算	通信	内存	IO
多进程/线程	√					√			√
高效语言改写	√					√			√
高性能存储	√								√
NUMA 绑核	√					√			√
混淆优化	√					√			√
高效内存分配器	√	√	√	√	√			√	
混合精度训练		√	√			√			
算子融合		√	√			√		√	
深度学习编译器		√	√			√		√	
高效通信算法				√			√		
梯度压缩				√			√		
计算通信掩藏				√			√		
异步更新				√			√		
高效优化器					√	√		√	
融合优化器					√	√		√	
ZeRO		√	√	√	√	√		√	
内存预分配		√	√		√			√	
垃圾回收策略调优		√	√		√			√	
负载均衡		√	√	√		√	√		
梯度累积				√	√	√	√	√	

Table 3 Optimization techniques used by some open-source tools for training performance optimization and their features

名称	使用优化技术	特性
NVIDIA apex	混合精度训练、融合算子、融合优化器	基于 NVIDIA 硬件
PyTorch FSDP/ FairScale	ZeRO 算法	引入额外通信成本,需结合计算通信掩藏与重计算使用
LightSeq/Light- Seq2	混合精度训练、融合算子、融合优化器、显存预分配、算子逻辑优化	基于 NVIDIA 硬件,针对 Transformer 类模型定制优化
Megatron	ZeRO 算法、混合精度训练、融合算子、激活重计算	提供多种模型训练并行策略,主要面向大语言模型训练
DeepSpeed	ZeRO 算法、混合精度训练、梯度压缩、高效优化器、融合优化器、计算通信掩藏	提供多种模型训练并行策略,主要面向大语言模型训练
FlashAttention	融合算子	主要提供 Attention 机制的高效实现
BladeDISC	深度学习编译器	支持多类型设备,对动态 Shape 进行了特殊优化

详细介绍目前常用的深度学习训练性能分析与可视化工具。

7.1 PyTorch Profiler

PyTorch Profiler 是 PyTorch 框架提供的性能数据采集工具^[48],可通过 Python 上下文管理器的方式使用相关 API。其可采集的信息包括:①模型算子执行时间;②模型执行过程中的张量形状;③模型算子堆栈信息;④张量内存分配与释放情况;

通过上述信息,用户可以:①定位执行较慢的算子;②

分析特定算子的调用关系,优化 Python 程序;③分析模型执行中内存的占用情况,优化内存分配逻辑等。

7.2 Nsight Systems

NVIDIA Nsight 是 NVIDIA 公司推出的帮助用户基于计算加速设备构建、调试、开发软件与分析性能的一组工具集^[49],其中 Nsight Systems 是一款较常用的系统级性能分析与可视化工具,其主要功能包括:①CPU 与 GPU 交互的可视化,完成 GPU 工作负载与 CPU 事件的关联;②跟踪 GPU 的活动,包括底层 IO(如 PCIe 吞吐量、NVLink 和 DRAM 的活动等)、SM 利用率、指令吞吐量等指标;③跟踪 GPU 工作负载,支持跟踪 CUDA API 与 CUDA 相关库的调用;④多节点性能分析,支持跨节点的性能通信分析与可视化。

Nsight Systems 可以提供较 PyTorch Profiler 更为底层的性能数据分析功能,但其仅支持 NVIDIA 公司的计算加速设备。

7.3 TensorBoard

TensorBoard 是深度学习框架 TensorFlow 推出的机器学习可视化工具^[50],其提供如下功能:①跟踪与可视化损失与准确率等指标;②可视化模型与其各层结构;③查看权重、偏置与其他张量随时间变化的直方图;④显示图片、文字与音频等数据。

用户可借助 TensorBoard 定位训练中的异常数据,方便分析问题根因。该工具可单独安装,因此能搭配多种深度学习框架使用。

8 优化案例——语音识别 Conformer 模型

Conformer 是经典的语音识别模型^[51],这里给出 Conformer 模型优化过程及效果,训练任务运行在华为 910B3 加速卡上。初始训练基线数据进程数 num_workers=1,批数据上限 max_tokens 为 10 240 帧,可通过如下优化提升单设备训练性能:①增大批数据上限,充分利用设备内存;②Add 与 LayerNorm 算子融合;③增加数据进程, num_workers=4;④融合 SGD 优化器。进一步通过修改梯度同步分桶大小,优化多设备训练性能,取得性能优化效果如表 4 所示,“+”表示在上一配置基础上新增项。

Table 4 Performance improvements in Conformer training

表 4 Conformer 模型训练性能提升情况

	效率(WPS)	配置	提升/%
单卡	1 556.6	基线, max_tokens=10240	/
	9 163.2	max_tokens=20480	488
	9 799.1	+Add 与 LayerNorm 融合算子	530
	11 948.7	+num_workers=4	668
	12 138.1	+融合 SGD 优化器	680
8 卡	56 009.2	基线, max_tokens=20480	/
	58 799.3	分桶 bucket_cap_mb=500	5
	79 785.4	+num_workers=4	42
	82 969.3	+计算均衡	48

9 结语

随着深度学习训练任务中参数与数据规模的快速增长,高效开展模型训练成为业界的研究热点,本文从系统角度介绍了通用深度学习训练的优化原理、技术与手段,并介绍了相关分析与可视化工具,旨在为读者建立深度学习训练性能优化的常规路径,帮助从业者开展相关工作。最近,大语言模型的出现又在深度学习领域掀起了一波新的研究热潮,但模型的超大规模带来了超高的训练成本,这是大语言模型从理论走向落地阻碍之一,面向大规模集群的分布式训练性能优化将是解决该问题的关键技术。

参考文献:

- [1] KRIZHEVSKY A, SUTSKEVER I, HINTON G E. ImageNet classification with deep convolutional neural networks[J]. Communications of the ACM, 2017, 60(6): 84-90.
- [2] GOODFELLOW I, BENGIO Y, COURVILLE A. Deep learning[M]. Boston: MIT Press, 2016.
- [3] ZHANG A, LIPTON Z C, LI M, et al. Dive into deep learning[M]. Cambridge: Cambridge University Press, 2023.
- [4] LI C, YAO Z, WU X. DeepSpeed data efficiency: improving deep learning model quality and training efficiency via efficient data sampling and routing[C]//Proceedings of the AAAI Conference on Artificial Intelligence, 2024: 18490-18498.
- [5] SHEN L, SUN Y, YU Z, et al. On efficient training of large-scale deep learning models: a literature review [DB/OL]. <https://arxiv.org/abs/2304.03589>, 2023.
- [6] LIN Q Y, CHEN Z G. The ignored I/O problem in LLM computility[J]. Communications of the CCF, 2024, 20(2): 21-26.
林清音, 陈志广. 大模型算力中被忽视的 I/O 问题[J]. 中国计算机学会通讯, 2024, 20(2): 21-26.
- [7] NGUYEN T T, TRAHAY F, DOMKE J, et al. Why globally reshuffle? revisiting data shuffling in large scale deep learning[C]//IEEE International Parallel and Distributed Processing Symposium, 2022: 1085-1096.
- [8] JIE F, ZHANG H, WANG J, et al. ASRDataset: a multi-granularity shuffle system for preparing large-scale ASR training data[C]//IEEE International Conference on Knowledge Graph, 2023: 69-76.
- [9] MICEKVICIUS P, NARANG S, ALBEN J, et al. Mixed precision training [DB/OL]. <https://arxiv.org/abs/1710.03740>, 2018.
- [10] KALAMKAR D, MUDIGERE D, MELLEMPUDI N, et al. A study of BFLOAT16 for deep learning training [DB/OL]. <https://arxiv.org/abs/1905.12322>, 2019.
- [11] BANNER R, HUBARA I, HOFFER E, et al. Scalable methods for 8-bit training of neural networks [DB/OL]. <https://arxiv.org/abs/1805.11046>, 2018.
- [12] WANG X, XIONG Y, WEI Y, et al. LightSeq: a high performance inference library for transformers[C]//Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2021: 113-120.
- [13] WANG X, WEI Y, XIONG Y, et al. LightSeq2: accelerated training for transformer-based models on gpus[C]//Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, 2022: 1-14.
- [14] MENGHANI G. Efficient deep learning: a survey on making deep learn-

- ing models smaller faster and better[J]. ACM Computing Surveys, 2023, 55(12): 1–37.
- [15] DAO T, FU D, ERMON S, et al. Flashattention: fast and memoryefficient exact attention with IO-awareness [C]//Advances in Neural Information Processing Systems, 2022: 16344–16359.
- [16] DAO T. FlashAttention-2: faster attention with better parallelism and work partitioning [C]//Vienna: International Conference on Learning Representations, 2024.
- [17] SHAH J, BIKSHANDI G, ZHANG Y, et al. FlashAttention-3: fast and accurate attention with asynchrony and low-precision [DB/OL]. <https://arxiv.org/abs/2407.08608>, 2024.
- [18] LI M, LIU Y, LIU X, et al. The deep learning compiler: a comprehensive survey[J]. IEEE Transactions on Parallel and Distributed Systems, 2020, 32(3): 708–727.
- [19] LATTNER C, AMINI M, BONDHUGULA U, et al. MLIR: scaling compiler infrastructure for domain specific computation [C]//IEEE/ACM International Symposium on Code Generation and Optimization, 2021: 2–14.
- [20] CHEN T, MOREAU T, JIANG Z. TVM: an automated end-to-end optimizing compiler for deep learning [C]//USENIX Symposium on Operating Systems Design and Implementation, 2018: 578–594.
- [21] ZHENG Z, YANG X, ZHAO P, et al. AStitch: enabling a new multi-dimensional optimization space for memory-intensive ML training and inference on modern SIMT architectures [C]//Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems, 2022: 359–373.
- [22] ZHENG Z, PAN Z, WANG D, et al. Bladedisc: optimizing dynamic shape machine learning workloads via compiler approach [C]//Proceedings of the ACM on Management of Data, 2023: 1–29.
- [23] ZHANG Z C. Optimization of distributed training strategies for deep learning networks [D]. Hangzhou: Zhejiang University, 2021.
张泽超. 深度学习网络分布式训练方案 研究与性能优化 [D]. 杭州: 浙江大学, 2021.
- [24] GIBIANSKY A. Bringing hpc techniques to deep learning [EB/OL]. <https://andrew.gibiansky.com/blog/machine-learning/baidu-allreduce/>.
- [25] SERGEEV A, DEL BALSIO M. Horovod: fast and easy distributed deep learning in TensorFlow [DB/OL]. <https://arxiv.org/abs/1802.05799>, 2018.
- [26] WANG G, VENKATARAMAN S, PHANISHAYEE A, et al. Blink: fast and generic collectives for distributed ml [C]//Proceedings of the Conference on Machine Learning and Systems, 2020: 172–186.
- [27] LIN Y, HAN S, MAO H, et al. Deep gradient compression: reducing the communication bandwidth for distributed training [C]//Vancouver Proceedings of the International Conference on Learning Representations, 2018.
- [28] GUO J, LIU W, WANG W, et al. Accelerating distributed deep learning by adaptive gradient quantization [C]//Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing, 2020: 1603–1607.
- [29] YAN G, LI T, HUANG S L, et al. AC-SGD: adaptively compressed SGD for communication-efficient distributed learning [J]. IEEE Journal on Selected Areas in Communications, 2022, 40(9): 2678–2693.
- [30] AWAN A A, HAMIDOUKHE K, HASHMI J M, et al. S-caffe: co-designing MPI runtimes and caffe for scalable deep learning on modern GPU clusters [C]//ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, 2017: 193–205.
- [31] ZHANG H, ZHENG Z, XU S, et al. Poseidon: an efficient communication architecture for distributed deep learning on GPU clusters [C]//USENIX Annual Technical Conference, 2017: 181–193.
- [32] SHI S, CHU X, LI B. MGWFBP: merging gradients wisely for efficient communication in distributed deep learning [J]. IEEE Transactions on Parallel and Distributed Systems, 2021, 32(8): 1903–1917.
- [33] ZHENG S, MENG Q, WANG T. Asynchronous stochastic gradient descent with delay compensation [C]//Proceedings of the International Conference on Machine Learning, 2017: 4120–4129.
- [34] MISHCHENKO K, BACH F, EVEN M, et al. Asynchronous SGD beats minibatch SGD under arbitrary delays [C]//Advances in Neural Information Processing Systems, 2022: 420–433.
- [35] CHEN K, HUO Q. Scalable training of deep learning machines by incremental block training with intra-block parallel optimization and block wise model update filtering [C]//Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing, 2016: 5880–5884.
- [36] CHEN K, DING H, HUO Q. Parallelizing adam optimizer with block-wise model update filtering [C]//Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing, 2020: 3027–3031.
- [37] KINGMA D P. Adam: a method for stochastic optimization [C]//San Diego: Proceedings of the International Conference on Learning Representations, 2015.
- [38] LOSHCILLOV I. Decoupled weight decay regularization [DB/OL]. <https://arxiv.org/abs/1711.05101>, 2017.
- [39] CHEN X, LIANG C, HUANG D, et al. Symbolic discovery of optimization algorithms [DB/OL]. <https://arxiv.org/abs/2302.06675>, 2024.
- [40] LIU H, LI Z, HALL D, et al. Sophia: a scalable stochastic secondorder optimizer for language model pre-training [C]//Vienna: International Conference on Learning Representations, 2024.
- [41] HIJMA P, HELDENS S, SCLOCCO A, et al. Optimization techniques for GPU programming [J]. ACM Computing Surveys, 2023, 55(11): 1–81.
- [42] YOU Y, GITMAN I, GINSBURG B. Large batch training of convolutional networks [DB/OL]. <https://arxiv.org/abs/1708.03888>, 2017.
- [43] HUO Z, GU B, HUANG H. Large batch optimization for deep learning using new complete layer-wise adaptive rate scaling [C]//Proceedings of the AAAI Conference on Artificial Intelligence, 2021: 7883–7890.
- [44] RAJBHANDARI S, RASLEY J, RUWASE O, et al. Zero: memory optimizations toward training trillion parameter models [C]//Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, 2020: 1–16.
- [45] CHEN T, XU B, ZHANG C, et al. Training deep nets with sublinear memory cost [DB/OL]. <https://arxiv.org/abs/1604.06174>, 2016.
- [46] ZHAO Y, GU A, VARMA R, et al. PyTorch FSDP: experiences on scaling fully sharded data parallel [J]. Proceedings of the VLDB Endowment, 2023, 16(12): 3848–3860.
- [47] PAUL I, HUANG W, ARORA M, et al. Harmonia: balancing compute and memory power in high performance GPUs [J]. ACM SIGARCH Computer Architecture News, 2015, 43(3): 54–65.
- [48] Meta. PyTorch profiler-PyTorch tutorials [EB/OL]. https://pytorch.org/tutorials/recipes/recipes/profiler_recipe.html.
- [49] NVIDIA. NVIDIA Nsight systems [EB/OL]. <https://developer.nvidia.com/nsight-systems>.
- [50] Google. TensorBoard: TensorFlow's visualization toolkit [EB/OL]. <https://www.tensorflow.org/tensorboard>.
- [51] GULATI A, QIN J, CHIU C, et al. Conformer: convolution-augmented transformer for speech recognition [C]//Shanghai: Proceedings of the Annual Conference of the International Speech Communication Association, 2020.